

Notes

1

`train.py` is used to train the Tetris agent. It creates the Tetris environment, initializes the neural network from `model.py`, and trains it over 2000 episodes.

Hyperparameters are defined in the beginning:

```
EPISODES = 2000
BATCH_SIZE = 512
GAMMA = 0.99
LR = 1e-3
MEMORY_SIZE = 30000
EPSILON_START = 1.0
EPSILON_END = 0.001
EPSILON_DECAY = 0.995
TARGET_UPDATE = 10
```

2

The training setup uses two networks:

1. `model`
2. `target_model`

Both are instances of the agent model. The first model is updated every training step, while `target_model` is updated only every 10 episodes.

I assume this is because the other model smooths out the training. A single run might get a lot of rewards or penalty and overestimate, meanwhile rewards over time can show correct training directions.

This is the model I described in my first idea, it outputs a score for the state.

3

For every active piece, the environment generates all possible actions and their resulting states. Each action is a (`rotation_index`, `column`) pair. For each valid action, `train.py` asks the environment what the board state would look like after making that move.

So the model is not directly choosing from raw button presses like left, right, rotate, or drop. Instead, it evaluates the board states and chooses the highest score.

4

Action selection uses epsilon-greedy exploration.

With probability `epsilon`, the agent chooses a random valid action. Otherwise, it sends all candidate next states through the model and picks the action with the highest score, like in actual usage.

The epsilon value slowly decreased, meaning the agent goes from choosing completely random options to more and more deterministic options.

5

A selected action is then applied and the state transition is stored in memory. In addition, the reward, final game state (game over) and next states are put with the transition.

6

Once the replay memory has at least `512` samples, the script trains on a random batch.

The target scores are initialized as the rewards for the transition.

If the game doesn't end, then the next possible moves are evaluated and their scores re added.

So the target is:

```
target = reward + gamma * max_future_q
```

7

The script also keeps track of training progress. Every `10` episodes it prints a short progress line. Every `100` episodes it saves:

1. the model weights to `tetris_dqn.pt`
2. training metrics to `training_metrics.csv`
3. plots to `training_plot.png`

For score and lines, it also adds a `50` episode moving average when there is enough data.